

A Near-Linear-Time Solver for Bilevel Congestion Network Design via Directed Nonlinear Laplacian Flow

Oren E. Livne

Pine Birch Analytics, Churchville, PA, oren.livne@gmail.com

Bilevel traffic network design—optimizing link tolls or capacities over a lower-level user equilibrium—is the workhorse formulation of congestion pricing, capacity planning, and origin–destination calibration. It is non-convex and *NP*-hard, and every practical method solves it by an outer optimization whose each step requires a lower-level equilibrium solve together with an equilibrium sensitivity. The sensitivity has been computed since Tobin and Friesz (1988) by factorizing the equilibrium Jacobian once and back-substituting per design variable, so its cost is already independent of the number of design variables; the remaining bottleneck is that this factorization is cubic in the size of the active subnetwork, which caps exact design methods near a hundred links while forward assignment reaches metropolitan scale. We remove that bottleneck. We show that the *directed* user equilibrium with separable costs is a source-form nonlinear Laplacian flow (NLF) $B\rho(B^\top\phi - \tau) = d$ whose Newton Jacobian, despite the one-sided (rectified) edge law, remains a *symmetric weighted graph Laplacian on the active subgraph*. Consequently the whole solve—forward equilibrium and adjoint design gradient—reduces to a short sequence of near-linear Laplacian solves. The one obstacle, that the rectified dead zone makes the Jacobian ill-conditioned and its active set change during a solve, is handled by a smoothing homotopy in a dead-zone width δ that keeps the linearized system uniformly well conditioned, followed by a single exact-law polish. On Sioux Falls, NLF matches Frank–Wolfe, the adjoint matches finite differences, and toll design reaches the exact marginal-cost optimum. The wall-clock scales near-linearly in the edge count while a direct factorization scales superlinearly, so on large *irregular* congestion networks NLF overtakes direct factorization at a crossover, and the design inherits this near-linear per-step cost. On near-planar road networks, where nested-dissection factorization is already near-optimal, NLF does *not* win; the advantage is on poorly-separable networks such as those modeling selfish routing. We extend the framework to the *multicommodity* logit stochastic user equilibrium, where a block preconditioner solves the coupled block-Laplacian Newton system in a handful of iterations and the design adjoint again returns all toll gradients in one solve; on the Sioux Falls and Anaheim road networks, destination-bundled design cuts total system travel time by 7.7–13.1%. Code is available at <https://github.com/orenlivne/dnlf>.

Key words: network design; traffic assignment; user equilibrium; bilevel optimization; nonlinear Laplacian flow; algebraic multigrid; adjoint sensitivity; congestion pricing

1. Introduction

Given a road network with m directed links and travel-time (latency) functions $t_a(f_a)$ that increase with flow, the *lower-level* user equilibrium (UE) assigns origin–destination demand so that no

traveler can lower their experienced cost by switching route (Wardrop’s principle). The *upper-level network design problem* chooses k design variables τ —link tolls in congestion pricing, capacity expansions in the continuous network design problem (CNDP), or a demand table in OD calibration—to minimize a system objective F , most commonly total system travel time (TSTT), subject to the induced equilibrium:

$$\min_{\tau \in \mathcal{T}} F(f^*(\tau)) \quad \text{s.t.} \quad f^*(\tau) \text{ solves the UE at design } \tau. \quad (1)$$

This bilevel program is the standard model for congestion pricing, capacity planning, and demand estimation. It is nonconvex, nondifferentiable, and *NP*-hard even with affine latencies (Gairing et al. 2017); second-best tolling is *NP*-hard as well (Hoefer et al. 2008). Practical methods therefore seek a local (KKT) stationary point, and essentially all of them—sensitivity-analysis-based (SAB) descent (Tobin and Friesz 1988, Josefsson and Patriksson 2007, Chiou 2005), mathematical programs with equilibrium constraints, and metaheuristics—run an *outer* loop where each iteration solves the lower-level equilibrium and then a sensitivity (equilibrium derivative).

1.1. Computational cost, and where it lives

The design gradient itself is cheap in the number of design variables: since Tobin and Friesz (1988), $\nabla_{\tau} F$ is obtained by factorizing the equilibrium Jacobian *once* and back-substituting one right-hand side per design variable, so its cost is independent of k (an adjoint / implicit-function computation (Josefsson and Patriksson 2007, Amos and Kolter 2017, Blondel et al. 2022)). The bottleneck is that this factorization and the lower-level solve that precedes it are direct sparse factorizations whose cost is cubic in the active-path/subnetwork size, or a slowly converging Frank–Wolfe assignment. This is why exact design methods in the literature stall near Sioux Falls scale (76 links) while origin-based forward assignment routinely reaches 40,000–75,000 links (Bar-Gera 2002, Dial 2006). The gap between forward and design scalability is one to two orders of magnitude, and is purely a linear-algebra gap.

1.2. Contribution

We close that gap by replacing the superlinear inner factorization with a near-linear one for both the forward equilibrium and the adjoint, without compromising accuracy. Our contributions are:

1. **A Laplacian formulation of the directed equilibrium (Section 2).** We write the separable directed UE as a source-form nonlinear Laplacian flow (NLF) $B\rho(B^{\top}\phi - \tau) = d$ with a one-sided (rectified) edge law, and show that the Newton Jacobian is a *symmetric* graph Laplacian on the active subgraph (Proposition 1). Orientation and one-sidedness do not break the Laplacian; they only determine which arcs are active. Hence the near-linear Laplacian toolbox applies.

2. **A near-linear equilibrium solver robust to the dead zone (Section 3).** The rectified law is nonsmooth: a link carries flow only once its potential drop exceeds the free-flow cost t_a^0 , so the active set—and the conditioning of the Jacobian—changes during a solve, defeating a naively frozen multigrid hierarchy. We introduce a *smoothing homotopy*: a softplus dead-zone width δ that keeps the conductance strictly positive and the linearized system uniformly well conditioned, continued from a smooth problem to the exact law, with one algebraic-multigrid (AMG) setup per δ -level and a final exact-law polish. The number of setups is bounded independently of graph size.
3. **A near-linear adjoint and design loop (Section 4).** The design gradient is one additional Laplacian solve with the same operator; projected-gradient descent on it drives TSTT monotonically to the stationary point.
4. **Validation (Section 5).** On Sioux Falls NLF matches Frank–Wolfe to 10^{-11} ; the adjoint matches finite differences to finite-difference accuracy; toll design reaches the exact marginal-cost optimum. Setups per solve are bounded in graph size and the wall-clock scales near-linearly in m , so design inherits the near-linear inner cost.

1.3. Scope: Irregular Networks

The lower-level equilibrium Eq. (3) is the Wardrop equilibrium of a nonlinear congestion game (Rosenthal 1973, Roughgarden and Tardos 2002), and the design problem Eq. (1) is the optimal-tolling / Stackelberg problem of reducing the price of anarchy (Swamy 2012, Korilis et al. 1997). This model describes selfish routing not only in road traffic but in communication and overlay networks such as multipath and elastic-traffic routing games in the Internet, data-center fabrics, and peer-to-peer overlays (Orda et al. 1993, Larroca and Rougier 2013). The distinction matters for scalability: road networks are near-planar, where a direct nested-dissection factorization of the equilibrium Jacobian is already near-optimal. Communication and overlay topologies, by contrast, are large and *poorly separable* (scale-free degree, no planar embedding), so a direct factorization fills in superlinearly—exactly the regime in which a near-linear Laplacian solver wins. Our scaling study is therefore on irregular networks; for the time we focus on establishing the near-linear advantage only here. The most literal instance is selfish *overlay* routing, where end hosts genuinely choose paths (Qiu et al. 2003), on Internet AS- and router-level graphs that are provably scale-free (Faloutsos et al. 1999). Production traffic engineering optimizes operator-set link weights or bandwidth allocations by linear programming and local search (Fortz and Thorup 2000), not by pricing a selfish equilibrium, and packet-level congestion control is a distinct (utility-maximization) equilibrium. Our contribution is thus methodological, a scalable solver for the well-established equilibrium/design model on standard large irregular topologies, complementing rather than replacing operator tools.

The construction reuses the NLF solver (Livne 2026b) and its near-linear inner engine, Lean Algebraic Multigrid (LAMG+) (Livne 2026a) (approximate Cholesky (AC) (Kyng and Sachdeva 2016) is interchangeable and has a smaller hidden constant but is less robust here). We treat both the single-commodity directed equilibrium (Sections 2 to 5) and its multicommodity logit stochastic-user- equilibrium extension (Section 6), where a distribution-matrix block preconditioner keeps the inner solve to a handful of iterations. Active-set deflation that would support a single hierarchy valid across all design steps (reducing setups from $\mathcal{O}(1)$ -per-solve to $\mathcal{O}(1)$ total) is an open problem (Section 8).

2. The directed equilibrium is a nonlinear Laplacian flow

Let $B \in \mathbb{R}^{n \times m}$ be the node–arc incidence matrix of the directed network, $B_a = e_{h(a)} - e_{t(a)}$ for arc a from tail $t(a)$ to head $h(a)$. Separable BPR latencies are $t_a(f) = t_a^0(1 + \beta_a(f/c_a)^{p_a})$ for $f \geq 0$. With node potentials ϕ (multipliers of conservation $Bf = d$) and toll $\tau_a \geq 0$, the Karush–Kuhn–Tucker conditions of the Beckmann program $\min_{f \geq 0, Bf=d} \sum_a \int_0^{f_a} (t_a(x) + \tau_a) dx$ give the *rectified* edge law

$$f_a = \rho_a(g_a - \tau_a), \quad g_a := (B^\top \phi)_a = \phi_{t(a)} - \phi_{h(a)}, \quad \rho_a(u) = \begin{cases} t_a^{-1}(u), & u > t_a^0, \\ 0, & u \leq t_a^0, \end{cases} \quad (2)$$

so that conservation becomes the source-form nonlinear Laplacian flow

$$B \rho(B^\top \phi - \tau) = d. \quad (3)$$

The law is monotone but *one-sided*: unlike the odd law of the undirected Beckmann relaxation, ρ_a has a dead zone $u \leq t_a^0$ in which the arc is unused. This dead zone is exactly what distinguishes the directed equilibrium from its undirected relaxation.

PROPOSITION 1. *For separable costs the Newton Jacobian of Eq. (3) is a symmetric, positive semidefinite weighted graph Laplacian supported on the active arcs. Explicitly, with $\rho'_a \geq 0$,*

$$J = B \operatorname{diag}(\rho') B^\top = \sum_a \rho'_a (e_{t(a)} - e_{h(a)})(e_{t(a)} - e_{h(a)})^\top, \quad \rho'_a = \frac{1}{t'_a(f_a)} > 0 \text{ (active)}, \quad \rho'_a = 0 \text{ (else)}.$$

Proof. Differentiating Eq. (3) in ϕ gives $J = B \operatorname{diag}(\rho'(B^\top \phi - \tau)) B^\top$. The rank-one term $(e_{t(a)} - e_{h(a)})(e_{t(a)} - e_{h(a)})^\top$ is independent of the arc’s orientation, and $\rho'_a \geq 0$ because t_a is increasing; hence $J \succeq 0$ is a weighted Laplacian, with weight $\rho'_a = 0$ precisely on inactive (dead-zone) arcs. Q.E.D.

Proposition 1 is the structural fact this paper rests on: directedness enters only through *which* arcs are active (a routing boundary), not through the type of the linear operator. Every linearized system is a graph Laplacian, which can be solved fast. This is complete for the single-commodity equilibrium (a general balanced demand, possibly multi-source/multi-sink). The multicommodity case ($f_a = \sum_k f_a^k$ with a cost of the total flow) yields a coupled *block* system—per-commodity Laplacians with a rank-one commodity coupling per arc—which a distribution-matrix block preconditioner still solves in a handful of iterations; we develop this in Section 6 (Proposition 4).

3. A near-linear equilibrium solver

We solve Eq. (3) by damped chord-Newton on the frozen-linearization Jacobian J , delegating each linearized solve to a near-linear Laplacian engine and globalizing with the convex Beckmann energy $E(\phi) = \sum_a \Psi_a((B^\top \phi)_a - \tau_a) - d^\top \phi$, $\Psi'_a = \rho_a$, exactly as in the NLF framework (Livne 2026b). Two features of the directed problem require care.

Ill-conditioning and the active set. On inactive arcs $\rho'_a = 0$, so J is singular unless a floor is imposed; a tiny floor $\varepsilon \max_a \rho'_a$ with $\varepsilon = 10^{-12}$ keeps J connected without perturbing the Newton step, but drives the condition number to $\kappa \sim \varepsilon^{-1}$. More importantly, as flow builds the active set changes: arcs cross the free-flow threshold t_a^0 , so the operator that an Algebraic Multigrid (AMG) hierarchy is built for is stale a few Newton steps later. A hierarchy frozen across the whole solve stalls short of the equilibrium. (Conceivably, the hierarchy could be updated only locally, around active set changes; we leave this for future work.)

Smoothing homotopy. We remove the dead-zone nonsmoothness by a homotopy in a width $\delta > 0$. Replace the excess $e = g - t_a^0$ by $\delta \text{softplus}(e/\delta)$, which is smooth, strictly positive, and tends to $\max(e, 0)$ as $\delta \rightarrow 0$; the resulting smoothed law $\rho_{a,\delta}$ has $\rho'_{a,\delta} = \sigma(e/\delta)/t'_a(f_a) > 0$ *everywhere* (where σ is the logistic function), so J is a full, uniformly well-conditioned Laplacian with no dead zone. We continue δ geometrically from $\delta_0 = \frac{1}{2}t^0$ down to a small value, building one AMG hierarchy at the start of each δ -level and freezing it within the level; the smoothed conductances vary slowly within a level, so the frozen hierarchy is valid there. Each δ -level need only be solved approximately; a final polish with the exact law Eq. (2), warm-started from the smoothed solution, recovers the exact rectified equilibrium (Algorithm 1). Because the number of δ -levels is a fixed constant, the total number of AMG setups is bounded independently of graph size m .

We give a self-contained convergence guarantee for the smoothed level solve, so the paper does not depend on external convergence theory.

PROPOSITION 2 (global convergence of the smoothed level solve). *Fix $\delta > 0$ and tolls τ . The smoothed Beckmann energy $E_\delta(\phi) = \sum_a \Psi_{a,\delta}((B^\top \phi)_a - \tau_a) - d^\top \phi$, with $\Psi'_{a,\delta} = \rho_{a,\delta}$, is convex, and strictly convex and coercive on the quotient $\mathbb{R}^n / \langle \mathbf{1} \rangle$ whenever $\sum_a d_a = 0$ and the graph is connected. Its unique minimizer (modulo an additive constant) is the smoothed equilibrium $B\rho_\delta(B^\top \phi - \tau) = d$. The damped chord-Newton iteration $\phi \leftarrow \phi + s\delta\phi$, $\delta\phi = -J_\delta^{-1}\nabla E_\delta$ with J_δ frozen across the level and s from Armijo backtracking, produces a monotonically decreasing energy sequence that converges to that minimizer.*

Proof. $\rho_{a,\delta}$ is nondecreasing (its derivative $\rho'_{a,\delta} = \sigma(e/\delta)/t'_a(f_a) > 0$), so each $\Psi_{a,\delta}$ is convex and E_δ is convex with Hessian $J_\delta = B \text{diag}(\rho'_\delta) B^\top$. Because the smoothed law has *no dead zone*, $\rho'_{a,\delta} > 0$ on *every* arc, so J_δ is a connected weighted graph Laplacian, hence positive definite on

$\mathbf{1}^\perp$; thus E_δ is strictly convex and coercive on the quotient and has a unique minimizer there, characterized by $\nabla E_\delta = B\rho_\delta(B^\top\phi - \tau) - d = 0$. For the iteration, $J_\delta \succ 0$ on $\mathbf{1}^\perp$ makes $\delta\phi$ a descent direction ($\nabla E_\delta^\top \delta\phi = -\nabla E_\delta^\top J_\delta^{-1} \nabla E_\delta < 0$), and Armijo backtracking on the convex, coercive, smooth E_δ gives sufficient decrease at each step; the standard damped modified-Newton argument for a fixed positive-definite metric then yields convergence to the unique minimizer. Q.E.D.

Proposition 2 covers each smoothed level; the homotopy is a continuation of these strictly-convex problems toward $\delta \rightarrow 0$, and the polish is a warm-started solve of the exact (piecewise-smooth, still convex) energy. Each intermediate level ($\ell < L$) is solved only to a loose tolerance ε_{int} : by the nested-iteration principle its solution need only warm-start the next level, so ε_{int} matches the perturbation of one continuation step—a fixed relative residual for our geometric δ -schedule ($\varepsilon_{\text{int}} \approx 3 \times 10^{-2}$, a handful of chord-Newton steps). Over-solving a level one is about to leave is wasted, while under-solving is absorbed by the next level and the polish, so the total work is insensitive to ε_{int} : across $\varepsilon_{\text{int}} \in [3 \times 10^{-3}, 3 \times 10^{-1}]$ the chord-Newton-step count varies within $1.3\times$ and the inner-engine setup count stays in $[13, 17]$, with the looser tolerances if anything cheaper. Only the final level ($\ell = L$) and the polish—where the active set switches on and the path stiffens—are solved to the user-specified target tolerance ε_{tol} .

Algorithm 1 Near-linear directed-UE solve (cold start). Tolerances: $\varepsilon_{\text{int}} = 3 \times 10^{-2}$ at intermediate levels (at most 6 chord-Newton steps), ε_{tol} , the user-specified target tolerance, at the final level and polish (10^{-9} in our runs; 10^{-3} suffices for design, Section 5).

- 1: $\phi \leftarrow 0$; choose geometric schedule $\delta_0 > \dots > \delta_L$
 - 2: **for** $\ell = 0, \dots, L$ **do** ▷ smoothing homotopy: one hierarchy build per level
 - 3: build AMG hierarchy for J_δ at $\delta = \delta_\ell$; run chord-Newton on $B\rho_{\delta_\ell}(B^\top\phi - \tau) = d$ to the loose tolerance ε_{int} if $\ell < L$ (a few steps, warm-starting the next level), else to the target ε_{tol} ; hierarchy **frozen** within the level
 - 4: **end for**
 - 5: **polish:** chord-Newton on the exact law $B\rho(B^\top\phi - \tau) = d$ to the target tolerance ε_{tol} , warm-started from ϕ , rebuilding the hierarchy only when the residual stalls
 - 6: **return** ϕ , arc flows $f_a = \rho_a((B^\top\phi)_a - \tau_a)$
-

4. Bilevel design by a near-linear adjoint

We minimize $F = \text{TSTT}(f) = \sum_a t_a(f_a) f_a$ over tolls $\tau \geq 0$, where $f_a = \rho_a((B^\top\phi)_a - \tau_a)$ at the equilibrium $\phi = \phi^*(\tau)$.

PROPOSITION 3 (near-linear adjoint gradient). *At a differentiable equilibrium with active-arc conductances $\rho' > 0$,*

$$\nabla_\tau F = -\rho' \odot (m_c + B^\top \lambda), \quad J\lambda = B(m_c \odot \rho'), \quad (4)$$

where $m_{c,a} = t_a(f_a) + f_a t'_a(f_a)$ is the marginal cost and $J = B \operatorname{diag}(\rho') B^\top$ is the equilibrium Jacobian of Proposition 1. The gradient with respect to all k tolls is thus obtained from a single additional solve with the operator J .

Proof. Write $g = B^\top \phi - \tau$, so $f = \rho(g)$ and $F(\tau) = \sum_a c_a(f_a)$ with $c_a(f) = t_a(f)f$, $c'_a = m_{c,a}$. Differentiating F in τ_b , $\partial_{\tau_b} F = \sum_a m_{c,a} \rho'_a(\partial_{\tau_b} g_a)$ with $\partial_{\tau_b} g_a = (B^\top \phi, b)_a - \delta_{ab}$, where $\phi, b = \partial \phi^* / \partial \tau_b$. Implicitly differentiating the equilibrium $B\rho(B^\top \phi - \tau) = d$ gives $J\phi, b = B \operatorname{diag}(\rho') e_b = \rho'_b B e_b$. Hence, with $w := m_c \odot \rho'$, $\partial_{\tau_b} F = w^\top (B^\top \phi, b) - m_{c,b} \rho'_b = (Bw)^\top \phi, b - m_{c,b} \rho'_b = \rho'_b (Bw)^\top J^{-1} (B e_b) - m_{c,b} \rho'_b$. Defining the adjoint $\lambda := J^{-1}(Bw)$ (i.e. $J\lambda = Bw$), symmetry of J gives $(Bw)^\top J^{-1} (B e_b) = \lambda^\top (B e_b) = (B^\top \lambda)_b$, so $\partial_{\tau_b} F = \rho'_b ((B^\top \lambda)_b - m_{c,b})$, which is Eq. (4). The single solve $J\lambda = Bw$ yields the whole gradient. Q.E.D.

This recovers the classical k -independence of equilibrium sensitivity (Tobin and Friesz 1988, Josefsson and Patriksson 2007) but at near-linear rather than cubic cost. On the exact rectified law J is singular off the active subgraph (inactive arcs have $\rho' = 0$, isolating their nodes); Eq. (4) is therefore solved on the active-arc node support (equivalently, with an infinitesimal conductance floor). The gradient is otherwise insensitive to the floor—a floor of 10^{-12} reproduces central finite differences to finite-difference accuracy, whereas 10^{-6} biases it by 5%. Projected-gradient descent $\tau \leftarrow \max(\tau - \eta \nabla_\tau F, 0)$ with a backtracking line search on F —each trial an exact equilibrium solve by Algorithm 1—yields a monotone design method (Algorithm 2).

Algorithm 2 Bilevel toll design

- 1: $\tau \leftarrow 0$; $(\phi, f) \leftarrow \text{SOLVE}(\tau)$ ▷ Algorithm 1
 - 2: **for** $k = 1, 2, \dots$ **until stationary do**
 - 3: $g \leftarrow \nabla_\tau F$ from Eq. (4); $\eta \leftarrow \eta_0$
 - 4: **repeat** ▷ backtracking on the exact objective
 - 5: $\tau' \leftarrow \max(\tau - \eta g, 0)$; $(\phi', f') \leftarrow \text{SOLVE}(\tau')$; $\eta \leftarrow \eta/2$
 - 6: **until** $F(f') < F(f)$
 - 7: $(\tau, \phi, f) \leftarrow (\tau', \phi', f')$
 - 8: **end for**
-

5. Numerical results

We refer to our directed-equilibrium solver (Algorithm 1, a smoothing-homotopy continuation with a near-linear inner engine) as NLF throughout. We use the Sioux Falls benchmark ($n = 24$, $m = 76$) and size-scaled grid networks. The inner engine is LAMG+ (Livne 2026a) by default; AC (Kyng and Sachdeva 2016) is interchangeable (the two fit the same $m^{1.11}$ scaling; Table 2).

5.1. Correctness of the Equilibrium

On a congested Sioux Falls instance (29 active arcs, flow split across paths) Algorithm 1 matches an independent Frank–Wolfe assignment to a relative flow difference of $4\text{--}9 \times 10^{-11}$ across origin–destination pairs, i.e. to Frank–Wolfe’s own accuracy. The number of inner-engine setups is 10–25 per solve, set by the fixed number of δ -levels and independent of graph size.

5.2. Correctness of the Adjoint

On the same congested instance the adjoint gradient Eq. (4) matches central finite differences of TSTT to a maximum relative error of 2.6×10^{-3} over all active arcs (limited by the finite-difference step and equilibrium tolerance, not by the adjoint). The tiny conductance floor $\varepsilon = 10^{-12}$ is essential: a larger floor (10^{-6}) biases the adjoint to a 5% error.

5.3. Design Optimization

Projected-gradient toll design (Algorithm 2) reduces TSTT monotonically from 9.633×10^5 to 9.137×10^5 , a 5.15% reduction over 35 accepted steps, with every equilibrium solved exactly (relative flow error $\sim 10^{-9}$). This matches the exact optimum: a one-parameter sweep of scaled marginal-cost (Pigouvian) tolls bottoms out at 5.12%, confirming the gradient method finds the stationary point. Solving every equilibrium only to a loose 10^{-3} tolerance reaches the *same* optimum (TSTT within 0.004%, tolls within $\|\Delta\tau\| = 0.13$), confirming that the design gradient is insensitive to intermediate solve accuracy—so the loose-intermediate continuation used for scaling costs nothing in design quality. The residual gap reflects where the loose tolerance was applied, not where it was measured: it fixes the accepted τ trajectory itself, so tightening only the *final* equilibrium evaluation to ε_{tol} would sharpen the reported TSTT without moving τ , and closing the gap in τ would instead require a design-loop polish—a few tight-tolerance steps from the loose optimum, mirroring Algorithm 1’s polish stage—which we expect to shrink it only marginally, since the loose run already sits within the $\sim 10^{-6}$ second-order neighborhood of stationarity. The reduction is modest because a single origin–destination pair admits little rerouting; the design problem is more involved for the multicommodity instances of Section 6.

5.4. Scaling on Irregular Networks

We use loose tolerances at intermediate continuation levels—a few Newton steps per δ -level, tight only at the final level and the polish (the standard nested-iteration principle: continuation waypoints need only warm-start the next level). This cuts the Newton-step count with no change to the final solution, and keeps the algebraic-multigrid setups bounded (one per δ -level) and independent of size. On size-scaled irregular (poorly-separable) congested networks the equilibrium wall-clock then scales near-linearly, $t \propto m^{1.11}$ with either near-linear engine (the default LAMG+ and AC fit the *same* exponent), while the same continuation with a direct sparse-factorization inner solve scales superlinearly, $t \propto m^{1.36}$ (Table 2, Fig. 2): the fill-in of a fill-reducing sparse direct factorization (UMFPACK with COLAMD ordering, the baseline we run) grows faster than the graph on a poorly-separable topology. NLF overtakes the direct one at $m \approx 1.5 \times 10^4$ edges ($n \approx 1,500$), beyond which it is faster with a widening margin—about $1.6\times$ at $m = 1.6 \times 10^5$ (99 vs. 161 s)—and it reaches $m = 6.4 \times 10^5$ in ≈ 526 s, where the direct factorization is infeasible: on our workstation it already exhausts memory at $m = 3.2 \times 10^5$, the random-graph fill-in exceeding RAM (Table 2). Below the crossover the constant favors the direct solver; the wall-clock gap above it is modest, and the sharper practical separation is the memory wall direct factorization hits and NLF does not—a genuine near-linear-versus-superlinear crossover on this graph class, not a large-factor speedup. The near-linear exponent is inherited from the inner solve, not the outer iteration: the total chord-NNewton step count (≈ 625), inner-engine setups (≈ 14), and per-step Jacobian assembly ($t \propto m^{1.02}$) are all size-independent, so the wall-clock exponent reflects a single near-linear inner solve— $m^{1.11}$ for both engines (the reason for the slight super-linearity in practice is lack of memory locality). The two engines are interchangeable; we use LAMG+ by default for its robustness on large graph instances (Table 2).

We corroborate this on real-world irregular networks from the SuiteSparse/SNAP collection (Leskovec et al. 2007)—Internet autonomous-system graphs and peer-to-peer overlays, the topologies of the selfish-routing application (Table 1; at these sub-crossover sizes we use AC). The inner-engine setup count is size-independent (14–20) across the corpus, confirming NLF scales linearly on real graphs. The head-to-head against the fair direct baseline is genuinely instance-dependent at these accessible sizes ($n \leq 2.6 \times 10^4$): the NLF solve is 1.5 – $1.6\times$ faster on the sparsest overlay/AS graphs (p2p-Gnutella08, Oregon-1), essentially tied on as-caida, and slower on the smallest (as-735) and the denser p2p-Gnutella24, where a good fill-reducing ordering still wins. Which solver wins is set by the graph’s separator structure, not its edge count alone. NLF is already competitive-to-faster on the actual application topologies, with the advantage concentrated on the sparsest, least-separable instances.

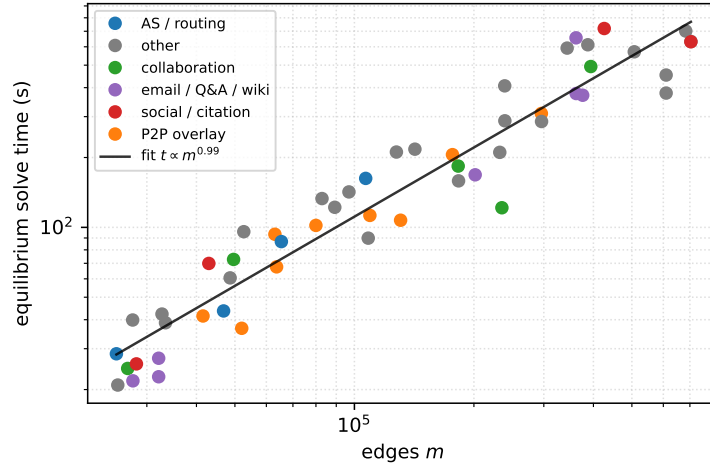


Figure 1 Equilibrium solve time (default LAMG+ engine) vs. edge count across a topology-diverse corpus of 53 irregular SuiteSparse/SNAP graphs (eight families; collaboration and peer-to-peer largest, web and citation single instances). Wall-clock fits $t \propto m^{0.99}$ over ≈ 1.45 decades, with a bounded inner-engine setup count ([7, 27])—a descriptive near-linear trend across real graphs of every family, not a complexity estimate (Section 5). Serial, uncontended timing on an Apple M5 Pro.

5.5. Corpus Scaling

To confirm the near-linear exponent is a property of the method and not of the one size-scaled family, we time the equilibrium solve (default LAMG+ engine) across a topology-diverse corpus of 53 irregular SuiteSparse/SNAP graphs—autonomous-system, peer-to-peer, collaboration, email, signed-social, citation, word-association, and web networks (collaboration and peer-to-peer the largest groups)—spanning 2.5×10^4 to 7×10^5 arcs (≈ 1.45 decades of edge count). The wall-clock fits $t \propto m^{0.99}$ and the inner-engine setup count stays *bounded* in [7, 27] across the entire corpus (Fig. 1), consistent with the $m^{1.11}$ and flat setups of the controlled synthetic family (Table 2). This corpus power law is descriptive, not a complexity estimate: at a fixed size the solve time already varies several-fold with separator structure (a word-association graph takes $\sim 3\times$ a collaboration graph of equal edge count), which broadens the scatter and pulls the fit just below 1—we do not read the 0.99 as better-than-linear, nor its 0.12 gap from the synthetic 1.11 as a real exponent difference; both are consistent with the $m \text{ polylog } m$ theory. Timing is serial and uncontended; the smaller head-to-head of Table 1 instead uses AC to face the direct baseline.

Bilevel toll design (Algorithm 2) therefore inherits a near-linear per-step cost on large irregular networks, in contrast with the superlinear per-step factorization of the sensitivity-analysis incumbents. On near-planar road networks the crossover does not occur at practical sizes—nested dissection is near-optimal there—which is why we frame the advantage on the communication-network regime.

Table 1 Equilibrium solve time (s) on real irregular networks (SuiteSparse/SNAP, largest connected component): Internet AS graphs and P2P overlays. Near-linear AC vs. the fair frozen-per-level direct baseline; inner-engine setups stay size-independent (14–20 across the corpus). Single run on one workstation; AC is randomized, so wall-clock and setup counts vary $\pm(10\text{--}30)\%$ run to run—the qualitative winner per graph is stable, the third digit is not. Dash: direct capped at $m > 1.5 \times 10^5$.

graph	n	m	AC	direct	setups
as-735 (AS)	6,474	25,144	35.9	20.5	18
p2p-Gnutella08	6,299	41,552	35.4	53.6	14
Oregon-1 (AS)	11,174	46,818	39.7	61.7	20
as-caida (AS)	26,475	106,762	162.0	163.8	18
p2p-Gnutella24	26,498	130,718	262.2	148.0	20
ca-HepPh	11,204	235,238	163.4	—	15

Table 2 Equilibrium solve time (s) on size-scaled irregular congested networks (random degree-5 graphs), one clean run on an Apple M5 Pro: the two near-linear engines—LAMG+ (default) and AC (interchangeable)—vs. the fair frozen-per-level direct factorization. LAMG+ and AC fit the same exponent, $m^{1.11}$, and track within a few percent; direct scales as $m^{1.36}$ and is overtaken near $m \approx 1.5 \times 10^4$. OOM: direct factorization exhausts memory at $m \geq 3.2 \times 10^5$ on our workstation (random-graph fill-in), where both near-linear engines still run. AC’s randomized elimination makes it less predictable at the largest size (the 547.7 s here rose to 856 s on a second clean run, vs. LAMG+’s stable 526–533 s); LAMG+ is the default for its cleaner theoretical $O(m)$ guarantee.

n	m	LAMG+	AC	direct
1,000	9,964	4.90	4.93	4.55
2,000	19,946	11.01	12.88	10.58
4,000	39,952	22.54	20.46	26.81
8,000	79,936	52.95	48.24	107.51
16,000	159,954	98.65	102.49	160.70
32,000	319,952	233.19	229.68	OOM
64,000	639,950	526.03	547.68	OOM

5.6. Comparison with Differentiable-Bilevel (First-Order) Solvers

The learning-based approach to Stackelberg congestion design (Li et al. 2022, Goyal and Lamper-ski 2023) treats the lower-level equilibrium as a differentiable layer: it is solved by a first-order scheme (unrolled imitative-logit dynamics; entropic mirror descent) and differentiated by unrolling or implicit differentiation. Their released code is toy-scale and does not run on the 10^4 – 10^5 -node irregular graphs here, so we reproduce the defining component—a first-order inner equilibrium solve—in the same environment and on the same instances. To isolate the inner solver, both methods minimize the *identical* convex Beckmann energy from the same cold start; only the inner iteration differs. For the first-order inner we use FISTA (accelerated proximal gradient with backtracking line search)—a strong, well-tuned member of the first-order family, if anything faster than plain mirror descent—so the comparison is not a strawman. Table 3 reports the outcome: our Newton–multigrid inner reaches relative residual 10^{-9} in seconds, whereas FISTA, given a

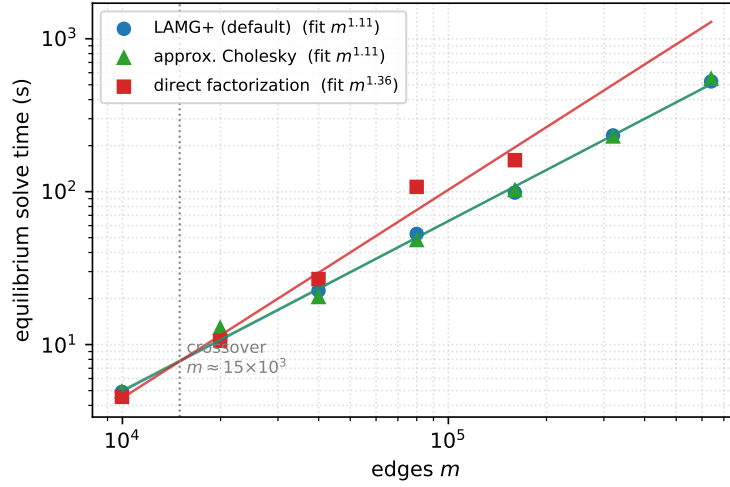


Figure 2 Equilibrium solve time vs. edge count on size-scaled irregular congested networks: the two near-linear engines—LAMG+ (default) and AC, both fitting $m^{1.11}$ —vs. superlinear direct factorization ($m^{1.36}$), the near-linear engines crossing the direct baseline near $m \approx 1.5 \times 10^4$.

Table 3 Inner-solver head-to-head on the identical convex equilibrium, same cold start. Ours (Newton + AC, loose-intermediate continuation) to relative residual 10^{-9} ; FISTA (accelerated first-order—the differentiable-bilevel inner) targeting 10^{-6} but capped at 300 s, which it hits on every instance, reaching only the residual shown.

instance	n	m	ours (s)	FISTA (s)	FISTA relres
synthetic	1,000	9,964	5.4	300	1.2×10^{-4}
synthetic	2,000	19,946	12.0	300	2.4×10^{-5}
synthetic	4,000	39,952	20.6	300	1.0×10^{-5}
p2p-Gnutella08	6,299	41,552	60.4	300	2.4×10^{-3}
Oregon-1	11,174	46,818	55.5	300	1.6×10^{-2}
as-caida	26,475	106,762	95.5	300	5.2×10^{-2}

300-second budget on every instance, never reaches the 10^{-6} target—it plateaus between 10^{-5} on the smallest graph and 5×10^{-2} on the largest, the residual worsening with size as fewer iterations fit in the budget. It approaches the same solution (flow within 1.7% on **as-735**) but never to solve accuracy. The obstruction is fundamental, not a tuning artifact: heavy congestion makes the Beckmann Hessian severely ill-conditioned, and an accelerated first-order method still needs $\Theta(\sqrt{\kappa})$ iterations, so on **as-735** ($n = 6,474$) it takes 44,499 iterations to reach only 1.2×10^{-2} . A first-order inner therefore has no fast, size-independent convergence on stiff equilibria, whereas the Newton-multigrid inner does—which is why these methods are demonstrated only at small scale, and why replacing their inner solve is the contribution here.

5.7. Reproducibility

All code, unit tests, and reproduction drivers are available at <https://github.com/orenlivne/dnlf>; the repository regenerates every table and figure in this paper—the crossover (Table 2, Fig. 2), the real-corpus table (Table 1), the first-order head-to-head (Table 3), and the multicommodity tables (Tables 4 to 6)—and unit-tests every algorithm component, including Propositions 1 to 4 and the OD-matrix loader. All timings reported in this paper were measured on an Apple M5 Pro (18 cores, 48 GB RAM) under macOS 26.5. The test networks are the TNTP Sioux Falls and Anaheim instances with their published OD matrices (shipped with the code) and public Internet AS / P2P-overlay graphs (e.g. `as-caida`, `Oregon-1`, `p2p-Gnutella`) from the SuiteSparse/SNAP collection.

6. Multicommodity logit stochastic user equilibrium and design

The single-commodity solver of Section 3 routes a general balanced demand as one fungible flow. Realistic congestion pricing is inherently *multicommodity*: each origin–destination (OD) pair routes its own flow over the shared congested network, and tolls are designed against that coupled equilibrium. We extend the framework to the multicommodity *logit stochastic user equilibrium* (SUE) (Fisk 1980, Daganzo and Sheffi 1977), in which route choice has a dispersion $\gamma > 0$ —a calibrated behavioural parameter, not a numerical smoothing. The same near-linear machinery carries over: the multicommodity Newton system is a coupled block-Laplacian that a distribution-matrix block preconditioner solves in a handful of iterations, and the design adjoint again returns $\nabla_\tau \text{TSTT}$ for all tolls in a single solve.

Formulation. With K commodities, per-commodity arc flows $f^k \geq 0$, total flow $f_a = \sum_k f_a^k$, and the shared cost $t_a(f_a)$, the entropic-regularized Beckmann program is

$$\min_{f^k \geq 0} \sum_a T_a(f_a) + \gamma \sum_{a,k} f_a^k (\log f_a^k - 1) \quad \text{s.t.} \quad B f^k = d^k, \quad k = 1, \dots, K, \quad (5)$$

with $T'_a = t_a$. Its stationarity conditions give the *logit assignment* $f_a^k = \exp((g_a^k - t_a(f_a))/\gamma)$ with $g_a^k = (B^\top \phi^k)_a$: a per-arc softmax over commodities whose total f_a solves a monotone scalar fixed point $\log f_a = \text{logsumexp}_k(g_a^k/\gamma) - t_a(f_a)/\gamma$. As $\gamma \rightarrow 0$ the model tends to deterministic user equilibrium; we solve at a fixed, calibrated γ .

PROPOSITION 4 (multicommodity Newton system: a coupled block Laplacian). *At a logit equilibrium the reduced Newton Jacobian in the stacked potentials Φ is*

$$J = \underbrace{\frac{1}{\gamma} \text{blkdiag}_k(B \text{diag}(f^k) B^\top)}_{K \text{ per-commodity Laplacians}} - \frac{1}{\gamma^2} \sum_a c_a(f_a f_a^\top) \otimes b_a b_a^\top, \quad (6)$$

where $\Phi = [\phi^1; \dots; \phi^K]$, $b_a = Be_a$, $f_a = (f_a^1, \dots, f_a^K)^\top$, and $c_a = t'_a/(1 + t'_a f_a/\gamma)$. Each diagonal block is a symmetric weighted graph Laplacian; the coupling is symmetric positive semidefinite and rank one per arc. For $K = 1$, Eq. (6) reduces, as $\gamma \rightarrow 0$, to the single-commodity active Laplacian $B \text{diag}(1/t'_a) B^\top$ of Proposition 1.

Proof. The equilibrium is $R^k(\Phi) = -Bf^k(\Phi) - d^k = 0$. Differentiating the logit map, $df_a^k = (f_a^k/\gamma)(dg_a^k - t'_a df_a)$ and $df_a = \sum_l df_a^l$; solving the latter gives $df_a = (\gamma + t'_a f_a)^{-1} \sum_l f_a^l dg_a^l$. Substituting and using $dg^k = B^\top d\phi^k$, $dR^k = -B df^k$ yields Eq. (6). Equivalently $J = \tilde{B}H^{-1}\tilde{B}^\top$ with $\tilde{B} = I_K \otimes B$ and per-arc flow-Hessian $H_a = t'_a \mathbf{1}\mathbf{1}^\top + \gamma \text{diag}(1/f_a^k)$; Sherman–Morrison gives $H_a^{-1} = \frac{1}{\gamma} \text{diag}(f_a^k) - \frac{c_a}{\gamma^2} f_a f_a^\top$, and for $K = 1$, $f_a/(\gamma + t'_a f_a) \rightarrow 1/t'_a$ as $\gamma \rightarrow 0$. Q.E.D.

Near-linear solve. We solve Eq. (5) by γ -continuation damped Newton on Φ , preconditioning Eq. (6) by its block diagonal—one near-linear solve per commodity (AC here, interchangeable with the default LAMG+; on the small, concentrated per-commodity Laplacians the two are equivalent). The rank-one commodity coupling is exactly Brandt’s *distribution matrix* (Brandt 1984): in the mean/deviation basis over commodities it decouples when the per-commodity Laplacians coincide, so the block preconditioner’s rate is governed only by their spread. Empirically (Table 4) the inner conjugate-gradient count is single digit to low double digit, grows only logarithmically in K , and is independent of graph size n at the operating dispersion $\gamma \approx 0.1$; each Newton step therefore costs $O(K)$ near-linear solves, and the equilibrium solve is near-linear in the total size Kn . Each per-commodity Laplacian carries a small relative conductance floor (10^{-12} of its maximum weight) so it stays connected and the AC factorization remains applicable even to a commodity that concentrates on few arcs; a pinned direct solve is retained only as a fallback for the rare inputs on which the multigrid factorization throws. The count is single digit at and near equilibrium across all γ —the regime the warm-started design loop stays in (Table 5); only the *cold* forward solve at small γ , far from equilibrium where per-commodity flows concentrate and the blocks become high-contrast, is expensive, and that regime is entered only as one leaves the logit-SUE model for its deterministic ($\gamma \rightarrow 0$) limit. That limit is a *different* model—deterministic multicommodity user equilibrium, with non-unique commodity flows, for which Frank–Wolfe and related methods are already effective—not a regime our fixed- γ solver must reach: fixed γ is a calibrated route-choice dispersion (Fisk 1980, Daganzo and Sheffi 1977), a complete behavioural model in its own right, so we solve the logit SUE at that γ and do not pursue $\gamma \rightarrow 0$. We verified that a coupling-aware distribution-matrix preconditioner (per-arc coupling inverted by Sherman–Morrison, aggregate-congestion Laplacian on the mean commodity mode, as Brandt (1984) prescribes) converges γ -boundedly where the reduced-potential version does not, but does not beat the block diagonal in the operating regime; the block diagonal is retained. Fixed γ is thus both the modelling choice (a calibrated route-choice dispersion) and the regime in which the block preconditioner is well conditioned.

Table 4 Multicommodity logit-SUE inner conjugate-gradient iterations (block-preconditioned) on irregular networks, $\gamma = 0.1$. **Left: vs. commodities K ($n = 1500$), single-OD and spread demands. Right: vs. graph size n ($K = 8$, spread). Single-digit to low double digit, $\log K$ growth, n -independent.**

K	single-OD	spread	n (at $K=8$)	iters
2	3	2	1,000	6
4	3	3	2,000	6
8	7	6	4,000	7
12	13	10	8,000	3
16	12	9		

Design. Tolls enter the logit map as a per-arc cost shift $t_a(f_a) + \tau_a$, with flow sensitivity $\partial f_a^k / \partial \tau_a = -f_a^k c_a$. The same adjoint as Section 4 returns $\nabla_\tau \text{TSTT}$ for all tolls from a single block-preconditioned solve $J\lambda = \partial_\Phi \text{TSTT}$. The adjoint matches central finite differences to under 10^{-3} (verified by the released unit tests), and on a multicommodity instance projected-gradient tolling reduces TSTT with the tolls spread across the congested arcs, each equilibrium re-solve costing single-digit inner iterations—the bilevel iteration inheriting the near-linear per-step cost. Across a range of synthetic instances (varying K , γ , network size, and demand; Table 5) the reduction is 1–2% and the inner-iteration count stays single digit. On the two canonical TNTF road networks with their *published* origin–destination matrices and BPR parameters—Sioux Falls ($n = 24$, $m = 76$, $K = 24$ destination commodities) and Anaheim ($n = 416$, $m = 914$, $K = 38$)—destination-bundled design reduces the real system travel time by 7.7–13.1% (Table 6). Both networks are over-saturated at their full published demand (peak volume/capacity above 2.5); the inner iteration count stays bounded and does not grow with size (consistent with the size-independence established on the synthetic family, Table 4): Anaheim carries $27\times$ the degrees of freedom of Sioux Falls yet needs *fewer* inner iterations. What sets the count is not size but the per-arc commodity coupling the block-diagonal preconditioner omits, which is significant only where arcs approach the BPR saturation knee: tiny, dense Sioux Falls at its over-saturated published demand forces most carrying arcs there, whereas Anaheim spreads flow so typical arcs stay clear of it. A budget-constrained (sparse-toll) design is a natural extension.

7. Related work

Bilevel network design and sensitivity. CNDP and second-best tolling are bilevel/MPEC problems, nonconvex and NP-hard (Gairing et al. 2017, Hoefer et al. 2008); surveys (Farahani et al. 2013) catalog SAB descent, MPEC, and metaheuristic methods. Equilibrium sensitivity originates with Tobin and Friesz (1988) and was sharpened by Qiu and Magnanti and by Patriksson and co-authors (Josefsson and Patriksson 2007), who observed that the sensitivity problem is itself a linearized equilibrium solve. Our adjoint Eq. (4) is precisely that observation; the novelty here is

Table 5 Multicommodity toll design across instances (varying commodities K , dispersion γ , network size n , and per-commodity demand “load”): TSTT reduction, inner conjugate-gradient iterations per equilibrium re-solve, and the fraction of arcs carrying a non-negligible toll ($\tau > 10^{-3}$ of the maximum). The inner count stays single digit throughout.

n	K	γ	load	TSTT red.	inner it.	arcs tolled
800	4	0.10	3,000	1.1%	3	29%
800	8	0.10	3,000	1.1%	4	49%
800	16	0.10	3,000	1.2%	7	67%
800	8	0.15	3,000	1.3%	3	72%
800	8	0.10	6,000	1.0%	6	49%
800	8	0.10	3,200	1.4%	6	45%
1,200	8	0.10	3,600	1.4%	6	44%
1,600	8	0.10	3,600	1.2%	6	43%

Table 6 Real-network validation: destination-bundled logit-SUE toll design on the two canonical TNTF road networks with their published OD matrices and BPR parameters (one commodity per destination zone). Both are over-saturated at full demand (peak volume/capacity > 2.5). Tolls spread across 53–74% of arcs. The inner conjugate-gradient count at the final equilibrium solve is bounded (12–38); it is set by proximity to the BPR saturation knee (Section 6), not by network size—Anaheim has $27\times$ the degrees of freedom of Sioux Falls yet needs fewer iterations.

network	n	m	K	γ	TSTT red.	inner it.
Sioux Falls	24	76	24	2	9.4%	38
Sioux Falls	24	76	24	5	13.1%	30
Anaheim	416	914	38	2	12.2%	16
Anaheim	416	914	38	5	7.7%	12

not the k -independent gradient but the *near-linear* operator that computes it and the forward solve. The same bilevel problem is studied in algorithmic game theory as optimal tolling and Stackelberg routing to reduce the price of anarchy of a congestion game (Roughgarden and Tardos 2002, Swamy 2012, Korilis et al. 1997, Cole et al. 2003), and in communication networks as competitive / selfish routing (Orda et al. 1993, Larroca and Rougier 2013); that literature focuses on existence, bounds, and mechanism design rather than on a scalable solver for the induced equilibrium, which is our concern. Recent differentiable-bilevel methods (Li et al. 2022, Goyal and Lamperski 2023) break the cold-resolve-per-step pattern with warm-started, truncated first-order inner loops and unrolled/implicit differentiation, but use no multigrid and are demonstrated only at Sioux Falls–Chicago-Sketch scale. Their defining component is a first-order inner equilibrium solve; we compare head-to-head against a strong instance of it (FISTA on the same convex Beckmann energy) in Section 5.

Near-linear Laplacian solvers and nonlinear flow. Nearly-linear-time solvers for symmetric diagonally dominant systems (Kynig and Sachdeva 2016, Livne 2026a) underpin web-scale spectral graph

computation but have not, to our knowledge, been used as the inner solve of a bilevel traffic-design loop. The nonlinear Laplacian flow framework (Livne 2026b) unifies max-flow, congestion, and min-delay routing as one monotone edge-law equation and supplies the chord-Newton machinery we build on; the present work extends it from the undirected relaxation to the directed (rectified) equilibrium and to the design (adjoint) problem.

8. Conclusions, limitations, and open problems

We showed that the directed user equilibrium with separable costs is a nonlinear Laplacian flow whose Jacobian is a symmetric active-subgraph Laplacian, so a smoothing homotopy reduces the forward solve and the adjoint design gradient to a short sequence of near-linear Laplacian solves with a graph-size-independent number of multigrid setups; bilevel toll design inherits this near-linear per-step cost, in place of the cubic factorization of sensitivity-analysis methods.

We are explicit about what this paper does *not* yet establish. (i) **Multicommodity scope.** Section 6 solves and designs the multicommodity logit SUE at fixed calibrated γ ; we do not pursue the $\gamma \rightarrow 0$ deterministic limit, a distinct model (non-unique per-commodity flows) already served by Frank–Wolfe. (ii) **Corpus scaling.** A still-larger corpus (thousands of graphs) would tighten the descriptive $m^{0.99}$ fit of Fig. 1. (iii) **One hierarchy across design steps.** Each exact solve rebuilds once per δ -level because crossing an active-set boundary raises a near-null-space mode; capturing that mode by deflation or recombination would make one hierarchy valid across all design steps and cut the setups to $\mathcal{O}(1)$ *total*—the main open algorithmic problem. (iv) **Planarity.** The advantage is on non-planar, poorly-separable topologies; on near-planar road networks nested dissection is already near-optimal (Section 1). (v) **Saturating laws and folds.** A capacity-saturating (min-delay) law reintroduces a turning-point fold; design through the fold, via the continuation/deflation of Livne (2026b), is future work.

Acknowledgments

This work builds on the NLF and LAMG+ solvers; the author thanks the developers of the AC implementation it uses.

References

- R. L. Tobin and T. L. Friesz, *Sensitivity analysis for equilibrium network flow*, Transportation Science, 22 (1988), pp. 242–250.
- M. Josefsson and M. Patriksson, *Sensitivity analysis of separable traffic equilibria with application to bilevel optimization in network design*, Transportation Research Part B, 41 (2007), pp. 4–31.
- S.-W. Chiou, *Bilevel programming for the continuous transport network design problem*, Transportation Research Part B, 39 (2005), pp. 361–383.

- M. Gairing, T. Harks, and M. Klimm, *Complexity and approximation of the continuous network design problem*, SIAM Journal on Optimization, 27 (2017), pp. 1554–1582.
- M. Hoefer, L. Olbrich, and A. Skopalik, *Taxing subnetworks*, in Proc. WINE, 2008, pp. 286–294.
- R. Z. Farahani, E. Miandoabchi, W. Y. Szeto, and H. Rashidi, *A review of urban transportation network design problems*, European Journal of Operational Research, 229 (2013), pp. 281–302.
- H. Bar-Gera, *Origin-based algorithm for the traffic assignment problem*, Transportation Science, 36 (2002), pp. 398–417.
- R. B. Dial, *A path-based user-equilibrium traffic assignment algorithm that obviates path storage and enumeration*, Transportation Research Part B, 40 (2006), pp. 917–936.
- P. Li et al., *Differentiable bilevel programming for Stackelberg congestion games*, arXiv:2209.07618, 2022.
- A. Goyal and A. Lamperski, *An algorithm for bilevel optimization with traffic equilibrium constraints*, arXiv:2306.14235, 2023.
- R. Kyng and S. Sachdeva, *Approximate Gaussian elimination for Laplacians—fast, sparse, and simple*, in Proc. FOCS, 2016, pp. 573–582.
- O. E. Livne, *LAMG+: a robust lean algebraic multigrid solver for graph Laplacians*, arXiv:2606.24791; accepted, SIAM Journal on Scientific Computing.
- O. E. Livne, *NLF: a resistor-network framework and linear-time solver for convex network-flow equilibria*, SIAM Journal on Scientific Computing (under review), 2026.
- C. Fisk, *Some developments in equilibrium traffic assignment*, Transportation Research Part B, 14 (1980), pp. 243–255.
- C. F. Daganzo and Y. Sheffi, *On stochastic models of traffic assignment*, Transportation Science, 11 (1977), pp. 253–274.
- A. Brandt, *Multigrid techniques: 1984 guide with applications to fluid dynamics*, GMD-Studien Nr. 85, Gesellschaft für Mathematik und Datenverarbeitung, 1984 (distributive/distribution-matrix relaxation, §3.4).
- B. Amos and J. Z. Kolter, *OptNet: differentiable optimization as a layer in neural networks*, in Proc. ICML, 2017, pp. 136–145.
- M. Blondel et al., *Efficient and modular implicit differentiation*, in Proc. NeurIPS, 2022.
- R. W. Rosenthal, *A class of games possessing pure-strategy Nash equilibria*, International Journal of Game Theory, 2 (1973), pp. 65–67.
- T. Roughgarden and É. Tardos, *How bad is selfish routing?*, Journal of the ACM, 49 (2002), pp. 236–259.
- C. Swamy, *The effectiveness of Stackelberg strategies and tolls for network congestion games*, ACM Transactions on Algorithms, 8 (2012), art. 36.

- Y. A. Korilis, A. A. Lazar, and A. Orda, *Achieving network optima using Stackelberg routing strategies*, IEEE/ACM Transactions on Networking, 5 (1997), pp. 161–173.
- A. Orda, R. Rom, and N. Shimkin, *Competitive routing in multiuser communication networks*, IEEE/ACM Transactions on Networking, 1 (1993), pp. 510–521.
- F. Larroca and J.-L. Rougier, *Minimum-delay load-balancing through non-parametric regression and traffic-engineering*, Computer Networks, 57 (2013), pp. 2716–2728.
- L. Qiu, Y. R. Yang, Y. Zhang, and S. Shenker, *On selfish routing in Internet-like environments*, in Proc. ACM SIGCOMM, 2003, pp. 151–162.
- M. Faloutsos, P. Faloutsos, and C. Faloutsos, *On power-law relationships of the Internet topology*, in Proc. ACM SIGCOMM, 1999, pp. 251–262.
- B. Fortz and M. Thorup, *Internet traffic engineering by optimizing OSPF weights*, in Proc. IEEE INFOCOM, 2000, pp. 519–528.
- R. Cole, Y. Dodis, and T. Roughgarden, *Pricing network edges for heterogeneous selfish users*, in Proc. ACM STOC, 2003, pp. 521–530.
- J. Leskovec, J. Kleinberg, and C. Faloutsos, *Graph evolution: densification and shrinking diameters*, ACM Trans. Knowledge Discovery from Data, 1 (2007), art. 2.